IIITDM Kancheepuram

Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram

LAN Collab

Technical Report & System Documentation

Course: Computer Networks (CS3004)

Submitted by:
Jatin Goyal (CS23B2045)
Vansh Singh (CS23I1028)

Submitted to:
Dr. Sanjeet Kumar Nayak
Department of Computer Science

Abstract

This document provides a comprehensive technical overview of LAN Collab, a robust, standalone, server-based communication application designed for exclusive operation over a Local Area Network (LAN). The system is architected as a one-stop solution for real-time collaboration in environments where internet access is unavailable, unreliable, or restricted.

It details the design, implementation, and protocols for the application's five core modules: multi-user video conferencing, server-side mixed audio conferencing, high-integrity screen sharing, group text chat, and secure file sharing. The report covers the client-server architecture, the specific network protocols employed (TCP, UDP, SSL), and the data serialization and media encoding formats (JSON, H.264, Mu-law). This document serves as both a user guide and a detailed technical reference for the system's source code, highlighting key algorithms, data flow, and security considerations.

Contents

Contents 1

1 Introduction 3
- 1.1 Project Overview and Goals . . . 3
- 1.2 Core Functional Requirements . . . 3
- 1.3 High-Level System Architecture . . . 4

2 Server Application Deep Dive 6
- 2.1 Overview and Initialization . . . 6
- 2.2 The Four Server Threads . . . 7
- 2.3 Server UI Panel . . . 8
- 2.4 Core: TCP Control Server (Port 9090) . . . 8
- 2.5 Core: UDP Media Server (Port 9091) . . . 9
- 2.6 In-Depth: Server-Side Audio Mixing . . . 9
- 2.7 Core: TCP File Server (Port 9100) . . . 11
- 2.8 Core: UDP Discovery Service (Port 9092) . . . 12

3 Client Application Deep Dive 13
- 3.1 Overview and Initialization . . . 13
- 3.2 Connection and Authentication . . . 14
  - 3.2.1 Step 1: LAN Discovery . . . 14
  - 3.2.2 Step 2: Secure TCP Connection . . . 14
- 3.3 Main User Interface . . . 15
- 3.4 In-Depth: Video Conferencing (H.264) . . . 17
- 3.5 In-Depth: Audio Conferencing (Mu-law) . . . 17
- 3.6 In-Depth: Screen Sharing (H.264 over TCP) . . . 18
- 3.7 In-Depth: File Sharing & Integrity . . . 18

4 Security & Data Protocols 20
- 4.1 Secure Serialization (JSON over Pickle) . . . 20
- 4.2 TCP Connection Security (SSL/TLS) . . . 20
  - 4.2.1 Certificate Generation (`generate_certs.py`) . . . 22
- 4.3 Authentication (Deterministic Meet Code) . . . 22

5 Project Source Files **23**
5.1 Key Dependencies 23
5.2 File Structure Breakdown 23
5.2.1 `lan_s29.py` - Server Application 23
5.2.2 `lan_c29.py` - Client Application 24
5.2.3 `auth_utils2.py` - Utility Module 24
5.2.4 Shared Utilities and Scripts 24

Chapter 1

Introduction

1.1 Project Overview and Goals

The primary objective of the LAN Collab project is to engineer a self-contained, multi-user communication platform that functions entirely within a Local Area Network, independent of any external internet services. In an era where data privacy and operational continuity are paramount, this application provides a vital solution for organizations (corporate, educational, or governmental) that require a secure, internal collaboration tool.

The project's design goals were:

- **Autonomy:** The system must be fully functional without an internet connection.
- **Performance:** Audio and video streams must have low latency to enable real-time conversation.
- **Security:** All sensitive communication must be encrypted, and data serialization must be secure.
- **Usability:** The client and server applications must have clean, intuitive, and modern GUIs.
- **Integrity:** Features like screen sharing and file sharing must prioritize data integrity.

1.2 Core Functional Requirements

The application's scope is defined by the successful implementation of five core modules:

1. **Multi-User Video Conferencing:** H.264 video capture and encoding (UDP) broadcast by the server.

2. **Multi-User Audio Conferencing:** Mu-law audio capture (UDP), server-side mixing, and broadcast.
3. **Slide & Screen Sharing:** H.264 screen capture streamed over reliable TCP for high clarity.
4. **Group Text Chat:** Real-time text messaging relayed by the server over TCP.
5. **File Sharing:** TCP file uploads to the server, with MD5-verified downloads available to all clients.

1.3 High-Level System Architecture

The system employs a classic **Client-Server architecture**. A single, authoritative server application acts as the central hub for all communication. Clients do not communicate directly with each other (peer-to-peer); all data is relayed through the server.

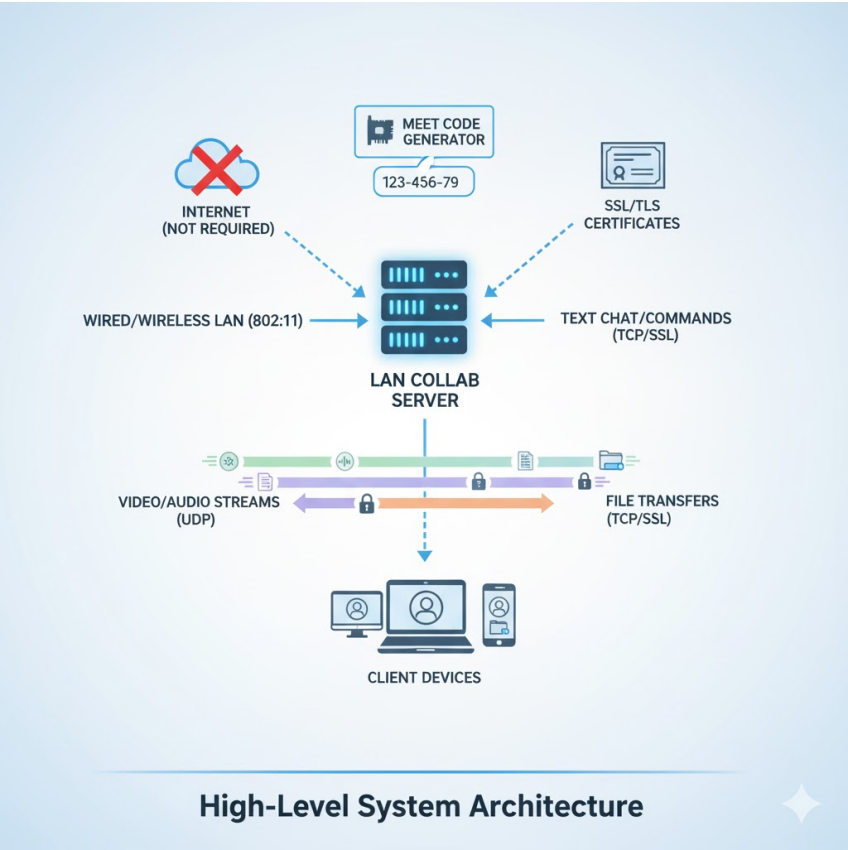

Figure 1.1: High-Level System Architecture

This architecture provides several advantages:

- **Simplicity:** Simplifies network logic, as clients only need to know the server's address.
- **Control:** Allows the server to manage session state, authentication, and user lists.
- **Efficiency:** Enables server-side processing, such as audio mixing, which reduces the computational load on clients.

The network communication is segregated by purpose, using four distinct ports to optimize for either low latency (UDP) or high reliability (TCP).

Figure 1.2: Network Protocol and Port Map

Chapter 2

Server Application Deep Dive

2.1 Overview and Initialization

The server application (`lan_s29.py`) is the backbone of the LAN Collab system. It is a multi-threaded PyQt6 application with a monitoring UI. Upon execution, the main server script:

1. Imports all necessary libraries.
2. Creates the main PyQt6 application and `ServerGUI` window.
3. Generates the deterministic 9-digit ”Meet Code” from the host’s MAC address.
4. Loads the SSL certificates (`server.crt`, `server.key`) for a secure TCP context.
5. Initializes all state-tracking variables (e.g., `tcp_clients`, `audio_buffers`).
6. Launches the four primary, non-blocking server threads.

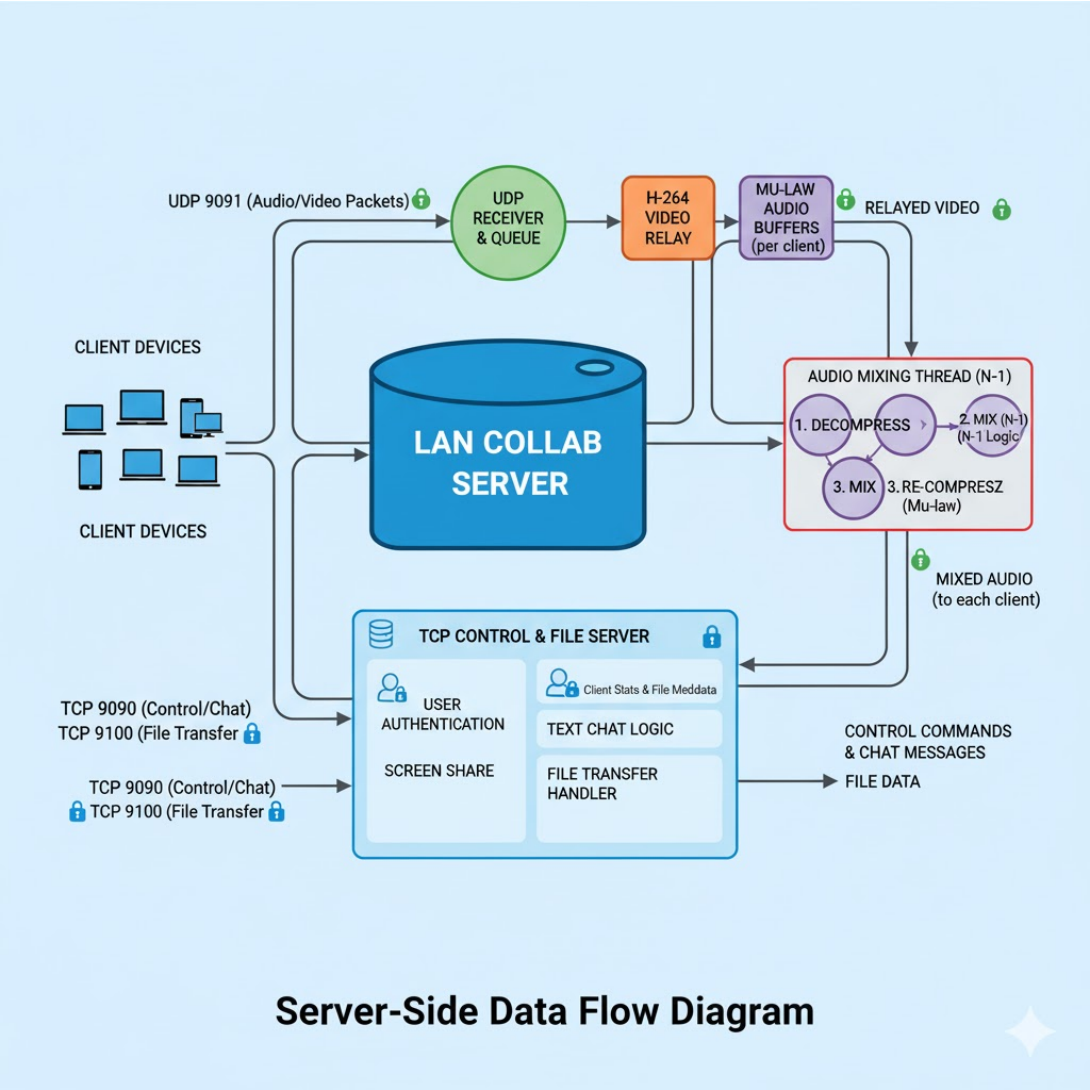

Figure 2.1: Server-Side Data Flow Diagram

2.2 The Four Server Threads

The server's functionality is split across four main threads for concurrent I/O:

- **TCP Control Server (Port 9090):** Manages connections, authentication, and reliable data (chat, commands).
- **UDP Media Server (Port 9091):** Ingests all low-latency audio and video packets from clients.
- **TCP File Server (Port 9100):** A dedicated, high-throughput channel for file uploads and downloads.
- **UDP Discovery Service (Port 9092):** Listens for LAN broadcasts from new clients and replies.

2.3 Server UI Panel

The server's GUI (from `server_ui.py`) provides "at-a-glance" information for the administrator:

- **Activity Log:** A real-time log of all major events (connections, errors, file transfers).
- **Connected Clients:** A list displaying the username and IP of every connected client.
- **Connection Info:** A read-only display of the server's LAN IP and the 9-digit Meet Code.

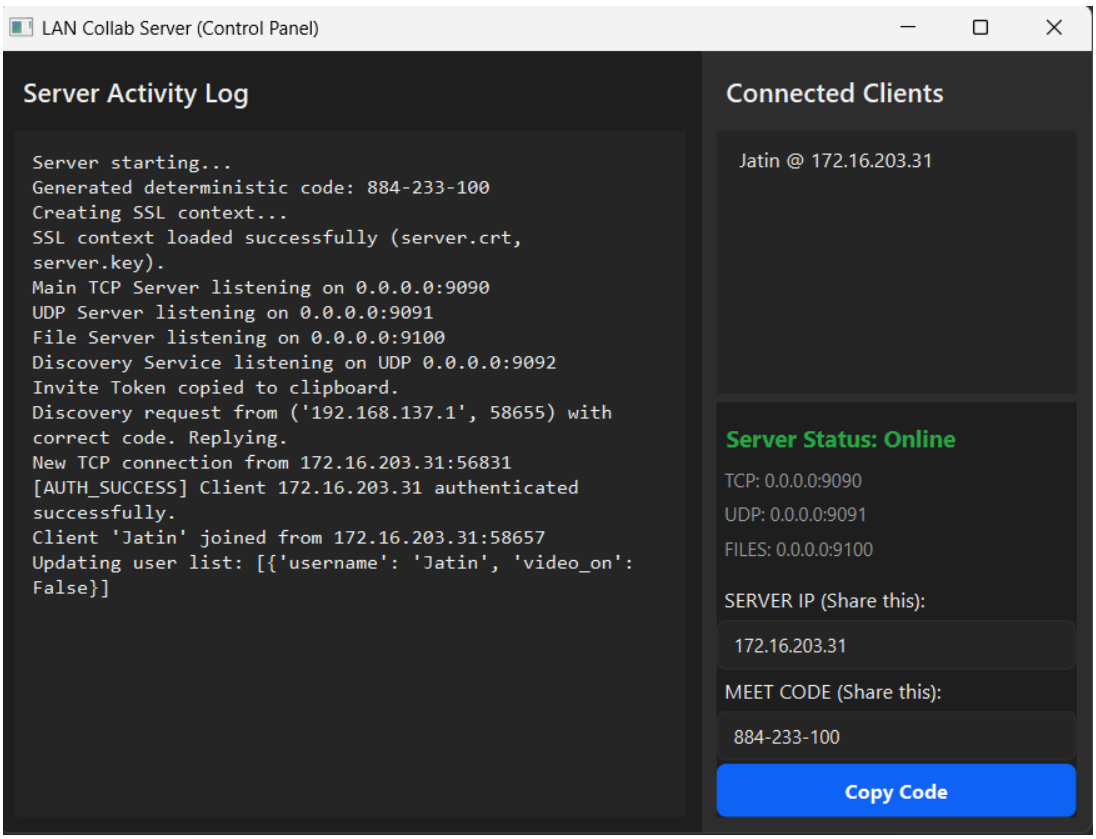

Figure 2.2: The Server Control Panel UI

2.4 Core: TCP Control Server (Port 9090)

This thread (from `server_networking.py`) is the "brain" of the server. It listens for new clients and spawns a new thread (`handle_tcp_client`) to manage each session. Its first task is authentication.

```
def handle_tcp_client(self, client_data):
    # ... [Initial setup] ...
    try:
        metadata = client_socket.recv(1024).decode('utf-8')
        parts = metadata.split(':')

```

```python
        # Expecting: JOIN:Username:UDPPort:MeetCode
        if not (parts[0] == 'JOIN' and len(parts) == 4):
            raise ValueError(f"Invalid JOIN format from {client_ip}")

        # ... [Parse username, port, code] ...
        client_meet_code = parts[3]

        # --- THE AUTHENTICATION CHECK ---
        if client_meet_code != self.meet_code:
            self.log_message(f"[AUTH_FAIL] Client {client_ip} used invalid code...")
            # ... [Send rejection message and close socket] ...
            return

        self.log_message(f"[AUTH_SUCCESS] Client {client_ip} authenticated.")
        # ... (rest of the client handling loop)
```

Listing 2.1: Server-side client authentication (from lan_s29.py)

2.5 Core: UDP Media Server (Port 9091)

The UDP server's `handle_udp_packets` thread ingests packets, validates the sender, and places them in a central `udp_packet_queue`. A separate `process_udp_queue` thread dequeues them:

- **If `'video_packet'`:** The H.264 data is immediately relayed to all other clients via UDP.
- **If `'audio'`:** The Mu-law data is placed in the sender's audio buffer, awaiting the mixing thread.

2.6 In-Depth: Server-Side Audio Mixing

This is the most complex part of the server (in `server_media.py`). A dedicated thread (`mix_and_send_audio`) runs on a high-precision timer to create a smooth audio stream.

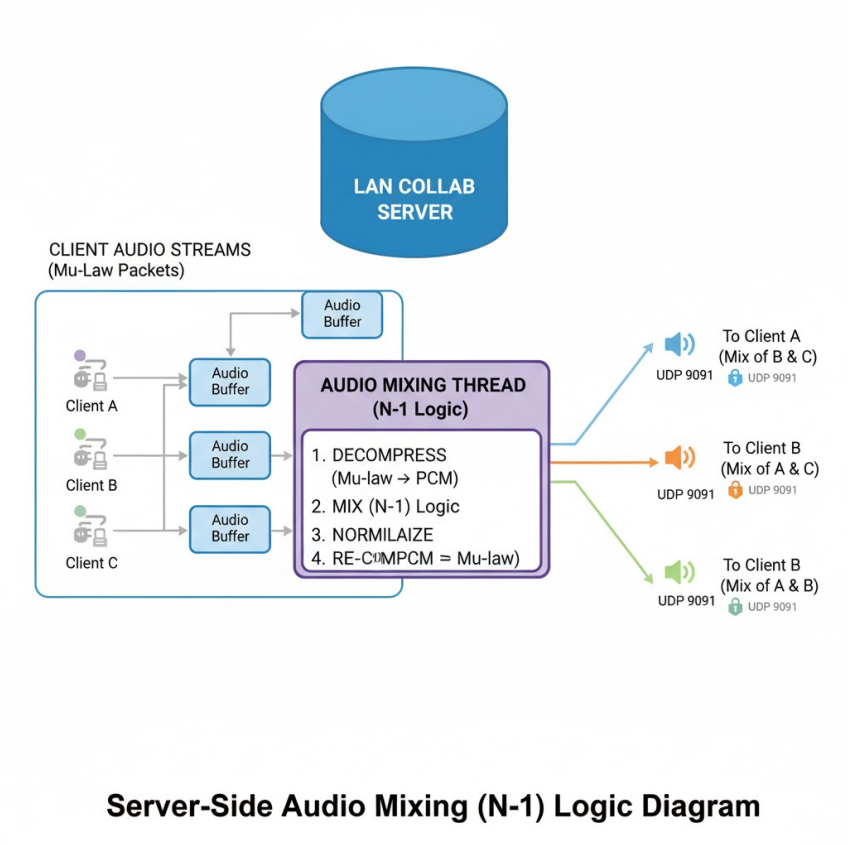

Figure 2.3: Server-Side Audio Mixing (N-1) Logic Diagram

The process for each ”tick” is:

1. **Gather Decompress:** Get one Mu-law audio chunk from each client’s buffer and decompress it to PCM.
2. **Mix (N-1):** For each client (Client A), create a ”mix buffer” by summing the PCM arrays of all *other* clients (to prevent echo).
3. **Normalize:** Apply a noise gate to the mix. If audible, normalize its volume (RMS) to a target level to prevent clipping.
4. **Re-compress Broadcast:** Compress the final, mixed PCM array back into Mu-law format and send the single packet via UDP to Client A.

This is repeated for every client, providing each with a custom mix of ”everyone else.”

```
# ... inside mix_and_send_audio() ...
for client_data in clients:
    target_addr = client_data['udp_addr']
    # ... [Initialize mix_buffer] ...

    for sender_addr, chunk_array in all_chunks.items():
        if sender_addr == target_addr:
            continue # Don't mix user's own audio
```

```
        # ... [Check chunk length] ...
        rms = np.sqrt(np.mean(chunk_array.astype(np.float32)**2))

        if rms > NOISE_THRESHOLD: # Noise Gate
            mix_buffer += chunk_array.astype(np.int32)
            # ... (Active speaker detection logic) ...

    # ... (Normalization logic) ...

    # --- Compress the final mix back to Mu-law ---
    compressed_mix = lin2ulaw_numpy(final_mix_int16)
    payload = safe_serialize({"type": "audio", "data": compressed_mix})

    self.udp_socket.sendto(payload, target_addr)
```

Listing 2.2: Core logic of server-side audio mixing (from server_media.py)

2.7 Core: TCP File Server (Port 9100)

This dedicated port (logic in `server_files.py`) handles large, reliable data transfers. The `handle_file_transfer` thread performs a handshake to determine the action.

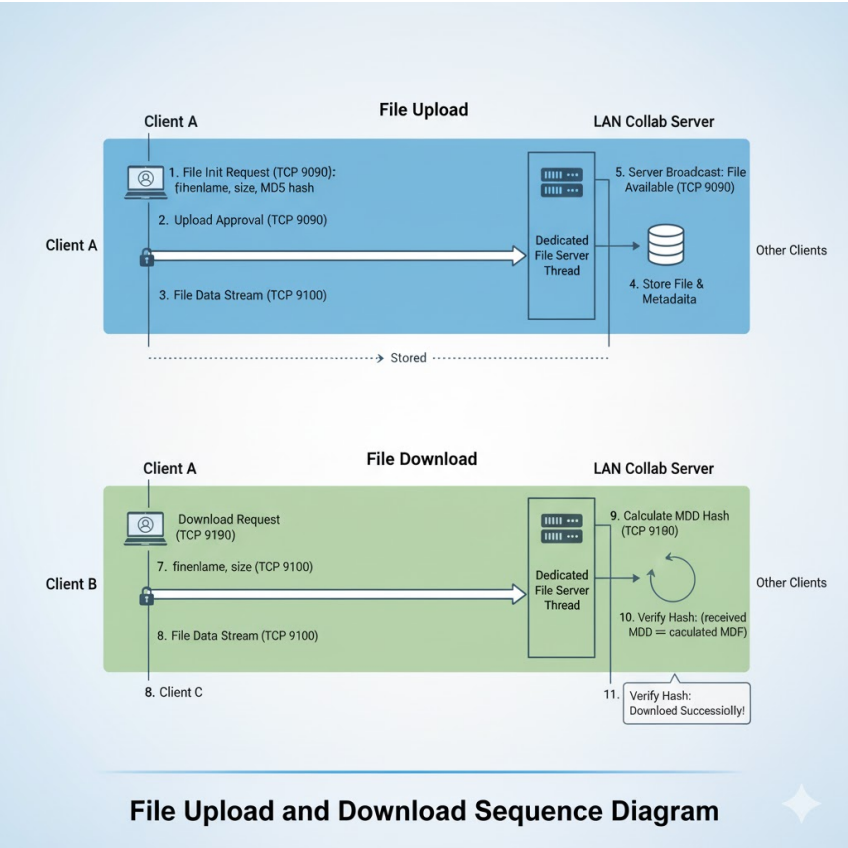

Figure 2.4: File Upload and Download Sequence Diagram

```python
def handle_file_transfer(self, conn):
    # ... [Read handshake data from conn] ...
    handshake_data = handshake_part.decode('utf-8').strip()
    action, transfer_id = handshake_data.split(':', 1)

    if action == "UPLOAD":
        with open(local_filepath, 'wb') as f:
            f.write(file_data_started)
            while True:
                chunk = conn.recv(4096)
                if not chunk: break
                f.write(chunk)

    elif action == "DOWNLOAD":
        # ... [Get local_filepath from transfer_data] ...
        with open(local_filepath, 'rb') as f:
            while True:
                chunk = f.read(4096)
                if not chunk: break
                conn.sendall(chunk)
```

Listing 2.3: File server handshake logic (from server_files.py)

2.8 Core: UDP Discovery Service (Port 9092)

This simple service (logic in `server_discovery.py`) listens for UDP broadcasts. If a packet contains the correct "Meet Code," the server replies directly to the sender with the main TCP control port (9090).

```python
def handle_discovery_requests(self):
    while self.running.is_set():
        try:
            data, addr = self.discovery_socket.recvfrom(1024)
            message = safe_deserialize(data)

            if message.get('type') == 'discover' and message.get('code') == self.meet_code:
                # ... [Get server_lan_ip] ...
                reply = safe_serialize({'type': 'discover_reply',
                                        'ip': server_lan_ip,
                                        'port': TCP_PORT})
                self.discovery_socket.sendto(reply, addr)
```

Listing 2.4: Discovery request handler (from server_discovery.py)

Chapter 3

Client Application Deep Dive

3.1 Overview and Initialization

The client application (`lan_c29.py`) is the user-facing component. It's a multi-threaded PyQt6 application. It handles media, networking, and UI rendering in separate threads.

Initialization sequence:

1. Display the `LoginDialog` (defined in `lan_c29.py`)
2. On success, initialize the main `ClientGUI` (defined in `lan_c29.py`).
3. Launch the connection thread (`connect_to_server`) (from `lan_c29.py`).
4. Once connected, start all media devices and background threads (from `lan_c29.py`).

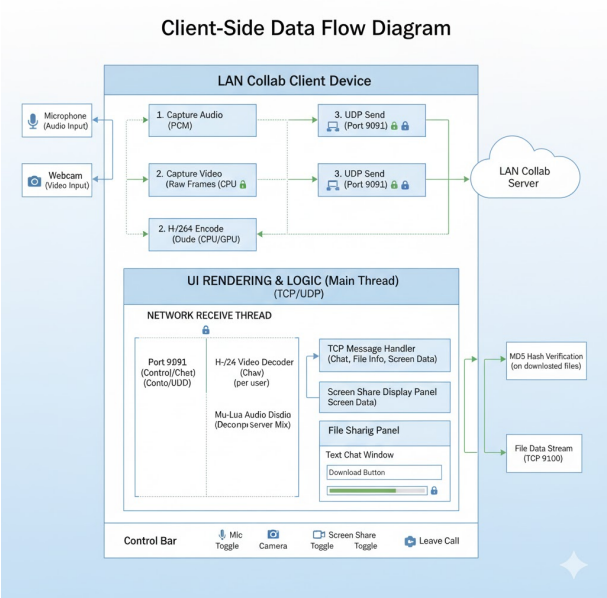

Figure 3.1: Client-Side Data Flow Diagram

3.2 Connection and Authentication

3.2.1 Step 1: LAN Discovery

The client first broadcasts a UDP "discover" message with the Meet Code. It listens for 2 seconds for a direct reply from the server.

```python
def connect_to_server(self):
    # ... [Create broadcast socket] ...
    msg = safe_serialize({'type': 'discover', 'code': self.meet_code})
    discover_socket.sendto(msg, ('<broadcast>', DISCOVERY_PORT))

    # ... (Listen for 2 seconds for a reply) ...

    if len(found_servers) == 0:
        raise socket.timeout("No server found with that code")
    else:
        self.server_host = found_servers[0]
        # ... (Proceed to connect_directly)

    except socket.timeout:
        self.signals.signal_discovery_failed.emit() # Fallback
```

Listing 3.1: Client-side discovery broadcast (from client_networking.py)

3.2.2 Step 2: Secure TCP Connection

Once the server's IP is known, the client initiates a secure (SSL) TCP connection and sends a `JOIN:` message to authenticate.

```python
def connect_directly(self):
    try:
        # ... [Create SSL context and wrap socket] ...
        self.tcp_socket = self.ssl_context.wrap_socket(unencrypted_socket,
                                                       server_hostname=self.server_host)
        self.tcp_socket.connect((self.server_host, TCP_PORT))

        # ... (Bind UDP port and get my_udp_port) ...

        join_message = f"JOIN:{self.username}:{my_udp_port}:{self.meet_code}"
        self.tcp_socket.send(join_message.encode('utf-8'))

        self.is_connected.set()
        # ... (Start all media threads) ...
```

Listing 3.2: Client-side authentication (from client_networking.py)

3.3 Main User Interface

The client UI (from `client_ui.py`) is an all-in-one ”mission control” for the user.

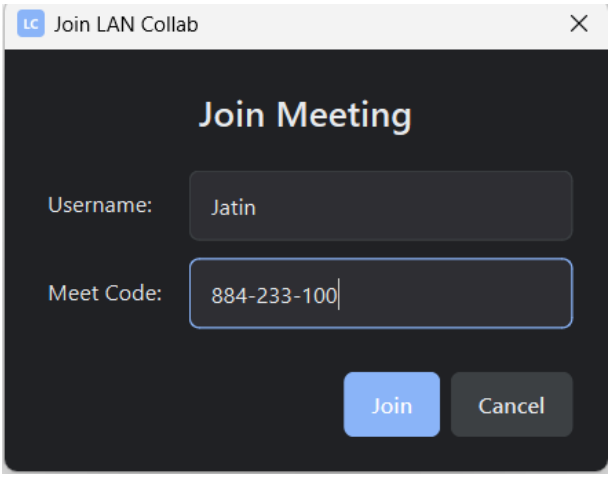

Figure 3.2: Client Login Dialog

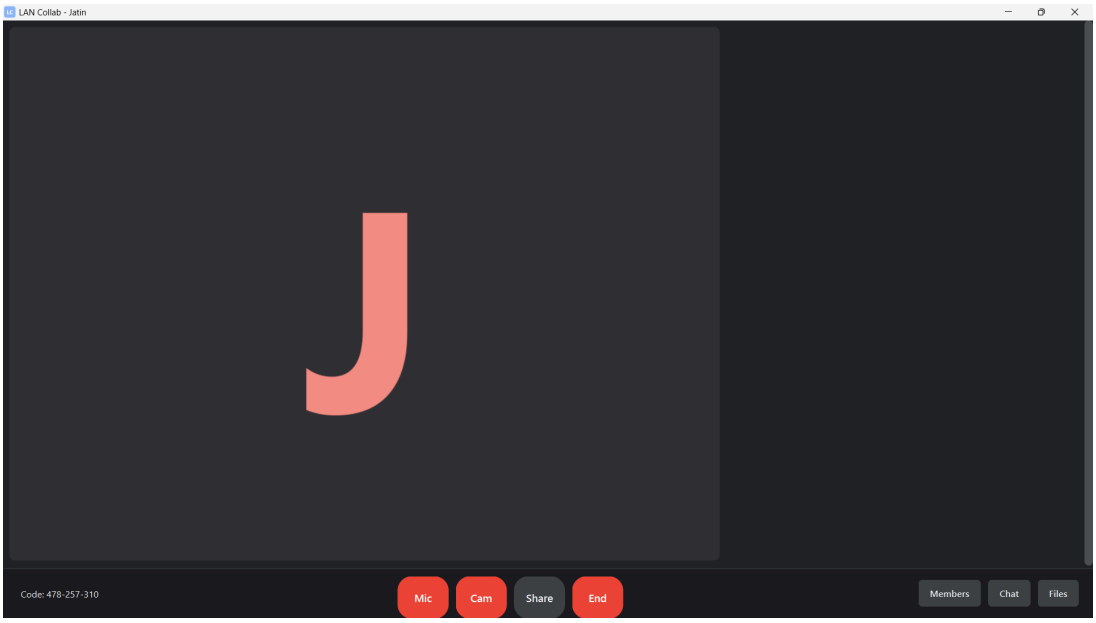

Figure 3.3: Main Client Interface (Single User)

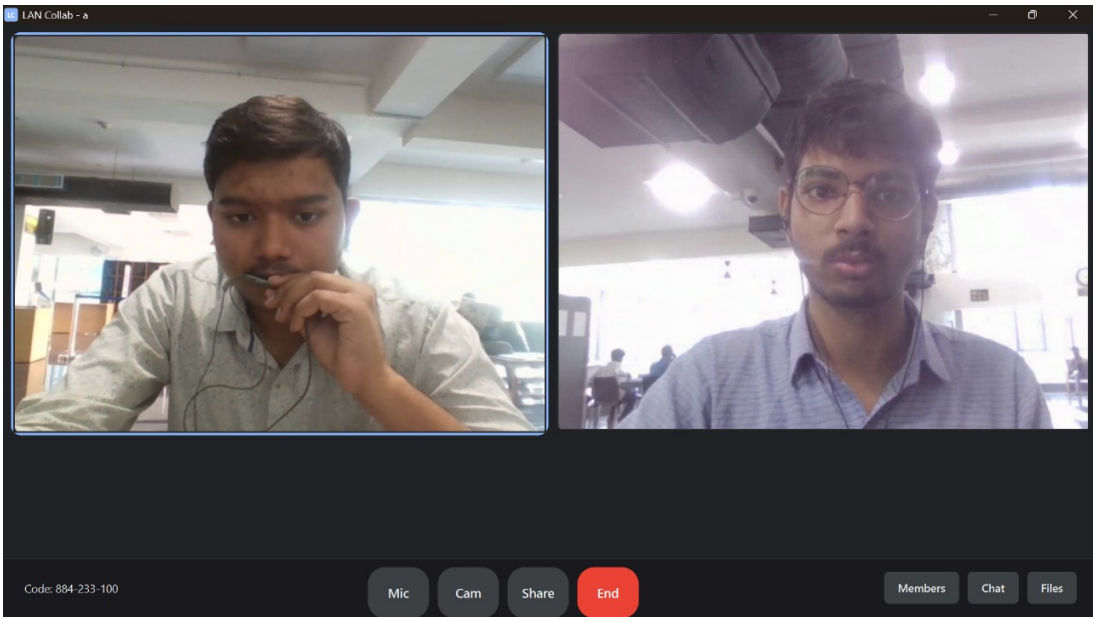

Figure 3.4: Testing: Multiple Users in Video Grid

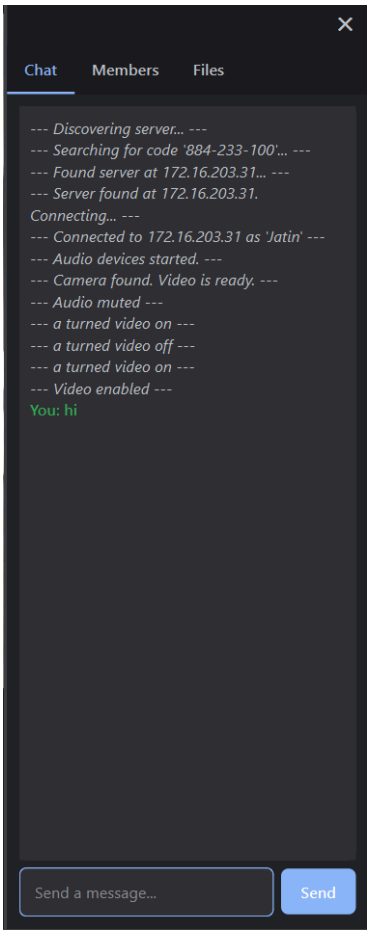

Figure 3.5: Testing: Active Text Chat Conversation

3.4 In-Depth: Video Conferencing (H.264)

Video is handled in a threaded process (in `client_media.py`):

1. **Capture & Encode (send_video thread):** Captures webcam frames, encodes to H.264 using `pyav`, and sends packets via UDP.
2. **Relay (Server):** The server relays these packets.
3. **Receive & Decode (receive_udp_data thread):** Receives H.264 packets, feeds them into a user-specific `pyav` decoder.
4. **Render (Main Thread):** The decoded `av.VideoFrame` is converted to a `QPixmap` and displayed in the UI via a Qt signal.

```
# ... inside send_video() thread ...
if frame_to_send is not None:
    try:
        av_frame = av.VideoFrame.from_ndarray(frame_to_send, format="bgr24")
        # ... [Set frame.pts] ...
        packets = self.video_encoder.encode(av_frame)

        for packet in packets:
            payload = {'type': 'video_packet', 'data': bytes(packet)}
            # ... [Serialize and send payload via UDP] ...
```

Listing 3.3: Client-side H.264 video encoding (from client_media.py)

3.5 In-Depth: Audio Conferencing (Mu-law)

Audio (from `client_media.py`) follows a distinct path:

1. **Capture & Compress (send_audio thread):** Reads `pyaudio` stream, compresses PCM data to Mu-law, and sends via UDP.
2. **Mix (Server):** Server performs N-1 mixing.
3. **Receive & Decompress (receive_udp_data thread):** Client receives the **single** mixed audio stream from the server and decompresses it back to PCM.
4. **Playback (receive_udp_data thread):** Writes the decompressed PCM data directly to the output `pyaudio` stream.

```
# ... inside send_audio() thread ...
if self.audio_enabled and self.audio_stream_in:
    data_pcm = self.audio_stream_in.read(CHUNK)
    data_pcm_array = np.frombuffer(data_pcm, dtype=np.int16)

    # --- Compress from 16-bit PCM to 8-bit Mu-law ---
```

```
    data_compressed = lin2ulaw_numpy(data_pcm_array)

    data_payload = safe_serialize({"type": "audio", "data": data_compressed
   })
    # ... [Send payload via UDP] ...
```

Listing 3.4: Client-side Mu-law audio compression (from client_media.py)

3.6 In-Depth: Screen Sharing (H.264 over TCP)

Screen sharing (from `client_screenshare.py`) uses TCP for integrity.

1. **Select Target:** User selects a window or screen from a dialog.
2. **Capture & Encode (screen_share_loop thread):** `mss` captures the screen, `pyav` encodes it to H.264.
3. **Send (TCP):** The H.264 packets are sent over the main TCP connection.
4. **Receive & Decode (Main Thread):** The main `handle_tcp_message` function receives, decodes, and displays the screen share.

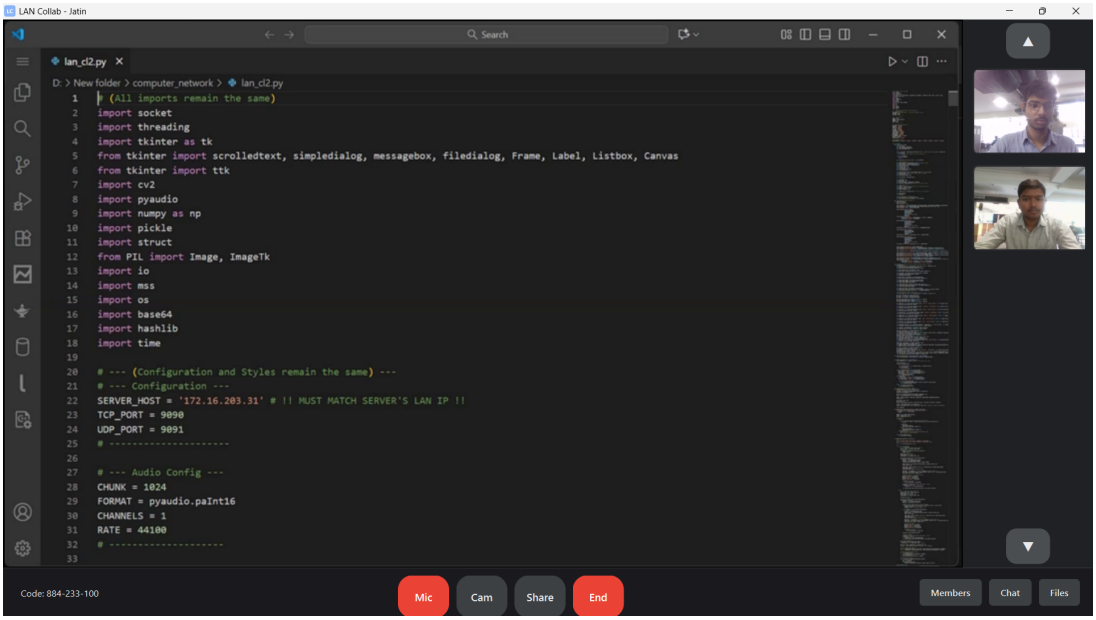

Figure 3.6: Viewing an active screen share

3.7 In-Depth: File Sharing & Integrity

File sharing (from `client_files.py`) is a multi-step, secure process.

1. **Offer:** Client selects a file, calculates its MD5 hash, and sends a `'file_init_request'` over TCP.

2. **Upload:** Server approves and tells the client to connect to the File Server (Port 9100) to stream the file.
3. **Notify:** Server broadcasts a `'file_available'` message to all other clients.
4. **Download:** Another client connects to Port 9100, sends a `DOWNLOAD` handshake, and streams the file.
5. **Verify:** The downloading client calculates the MD5 hash of the received file and compares it to the hash from the server to ensure integrity.

Figure 3.7: File Sharing Panel

```python
def hash_file_md5(self, filepath):
    hash_md5 = hashlib.md5()
    try:
        with open(filepath, "rb") as f:
            for chunk in iter(lambda: f.read(4096), b""):
                hash_md5.update(chunk)
        return hash_md5.hexdigest()
    # ... [Error handling] ...
```

Listing 3.5: Client-side MD5 hash verification (from $\text{lan}_c 29.py$)

Chapter 4

Security & Data Protocols

4.1 Secure Serialization (JSON over Pickle)

To prevent remote code execution vulnerabilities from `pickle`, this project uses a **JSON + Base64** protocol. All binary data (audio, video) is Base64-encoded before being inserted into a JSON object. This is both secure and language-agnostic.

```python
import json
import base64

BINARY_KEYS = ['data', 'chunk', 'frame'] # Keys with binary data

def safe_serialize(payload):
    try:
        payload_copy = payload.copy()
        for key in BINARY_KEYS:
            if key in payload_copy and isinstance(payload_copy[key], bytes):
                # Encode bytes to base64 string
                payload_copy[key] = base64.b64encode(payload_copy[key]).decode('utf-8')

        return json.dumps(payload_copy).encode('utf-8')
    # ... [Error handling] ...
```

Listing 4.1: Secure serialization function (from lan_s29.py / lan_c29.py)

4.2 TCP Connection Security (SSL/TLS)

All reliable TCP communication is encrypted using SSL/TLS. The server loads a `server.crt` and `server.key` to create a secure context. Both the Control Server (9090) and File

Server (9100) wrap their sockets with this context, encrypting logins, chat, screen sharing, and file transfers.

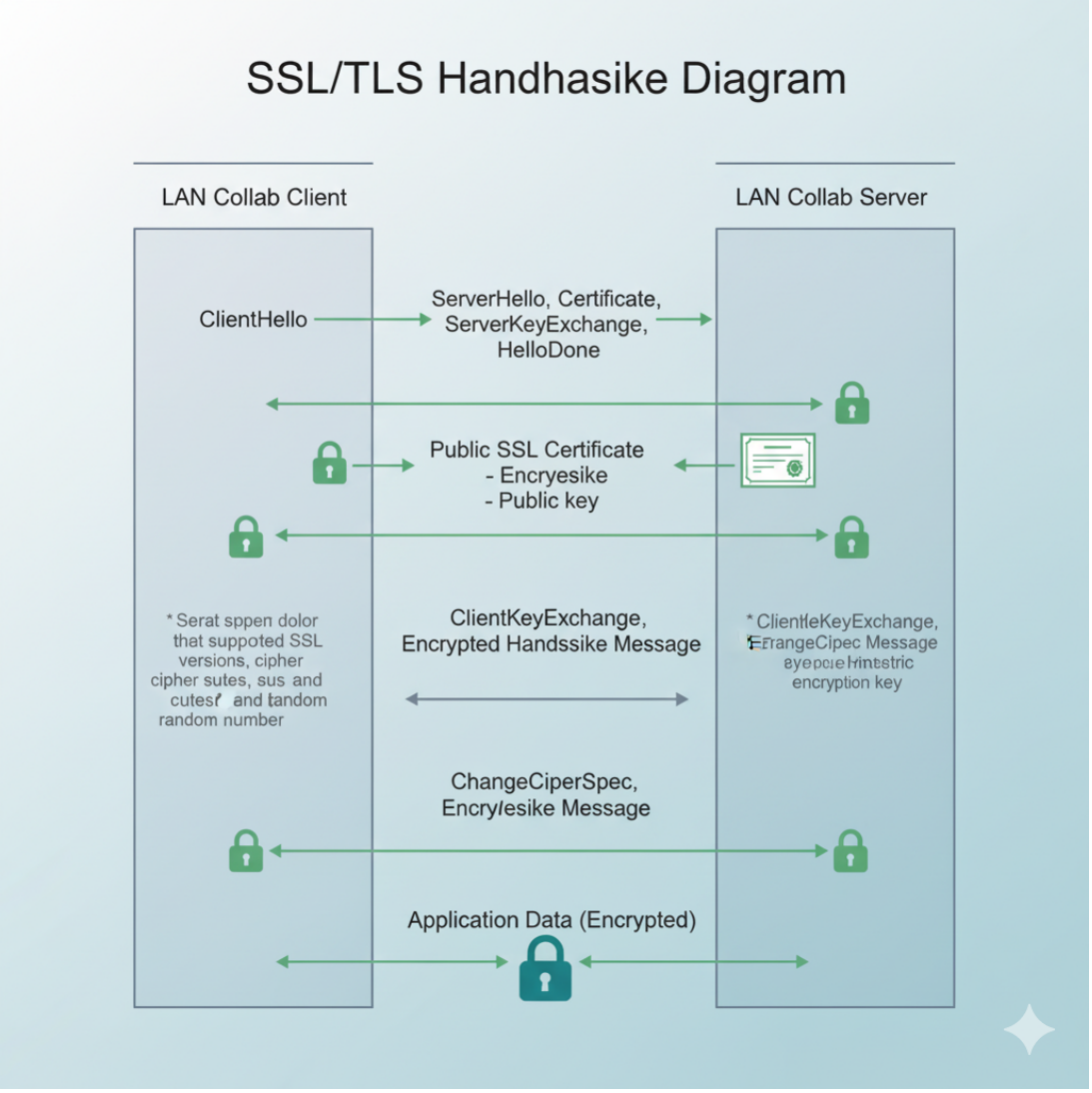

Figure 4.1: SSL/TLS Handshake Diagram

```python
import ssl

# ... inside ServerGUI __init__ ...
try:
    self.ssl_context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
    self.ssl_context.load_cert_chain(certfile="server.crt", keyfile="server.key")
    self.log_message("SSL context loaded successfully (server.crt, server.key).")
except Exception as e:
    # ... [Fatal error handling] ...

# ... inside start_tcp_server() ...
```

```python
try:
    # ... [Socket setup and bind] ...
    # --- Wrap the socket with SSL ---
    self.tcp_socket = self.ssl_context.wrap_socket(self.tcp_socket, server_side=True)
    self.log_message(f"Main TCP Server listening on {HOST}:{TCP_PORT}")
    # ... (start accept thread)
```

Listing 4.2: Server-side SSL socket wrapping (from lan_s29.py)

4.2.1 Certificate Generation (`generate_certs.py`)

A helper script, `generate_certs.py`, is provided to create the self-signed `server.key` (private) and `server.crt` (public) files. This only needs to be run once during initial server setup.

4.3 Authentication (Deterministic Meet Code)

Authentication is handled by a 9-digit "Meet Code" (e.g., `123-456-789`) generated from the server's permanent MAC address. This logic is in `auth_utils2.py`. This ensures the code is unique to the server hardware and persistent across reboots.

```python
import uuid
import random

def generate_deterministic_numeric_code():
    try:
        mac_num = uuid.getnode()
        random.seed(mac_num)
        code_num = random.randint(100_000_000, 999_999_999)

        # ... [Format number into 123-456-789] ...

        return f"{str(part1).zfill(3)}-{str(part2).zfill(3)}-{str(part3).zfill(3)}"
    # ... (Error handling)
```

Listing 4.3: Deterministic code generation (auth_utils2.py)

Chapter 5

Project Source Files

5.1 Key Dependencies

This project relies on several key open-source Python libraries.

Library	Purpose
`PyQt6`	For the client and server GUIs.
`pyaudio`	For cross-platform audio capture and playback.
`opencv-python (cv2)`	For webcam capture.
`pyav`	For high-performance H.264 video encoding and decoding.
`numpy`	For high-performance, low-level audio mixing.
`mss`	For fast and efficient screen capture.
`pygetwindow`	For enumerating application windows for screen sharing.
`json, base64, hashlib`	For secure data serialization and file integrity.
`ssl, socket, threading`	Core Python libraries for networking and concurrency.

Table 5.1: Key Project Dependencies

5.2 File Structure Breakdown

The project source code is contained in three main Python files, supported by security certificates.

5.2.1 `lan_s29.py` - Server Application

This is the main executable for the server. It contains all logic for the server's operation.

- **Class** `ServerGUI`**:** The main PyQt6 window class.
- **Core Functions:** `start_tcp_server`, `start_udp_server`, `start_file_server`, `start_discovery_service`.

- **Thread Targets:** `handle_tcp_client`, `handle_udp_packets`, `process_udp_queue`, `mix_and_send_audio`, `handle_file_transfer`, `handle_discovery_requests`.
- **Helpers:** `safe_serialize`, `lin2ulaw_numpy`, `ulaw2lin_numpy`, `broadcast_message`.

5.2.2 `lan_c29.py` - Client Application

This is the main executable for the client. It contains all logic for the client's operation.

- **Class** `LoginDialog`**:** The initial login window.
- **Class** `ClientGUI`**:** The main application window class.
- **Connection Logic:** `connect_to_server`, `connect_directly`.
- **Thread Targets:** `receive_tcp_data`, `receive_udp_data`, `send_video`, `send_audio`, `screen_share_loop`, `client_upload_file`, `client_download_file`.
- **Helpers:** `safe_serialize`, `av_frame_to_qpixmap`, `hash_file_md5`.

5.2.3 `auth_utils2.py` - Utility Module

This is a shared helper file imported by both the client (for IP lookup) and the server.

- **Function** `generate_deterministic_numeric_code()`**:** Uses the server's MAC address to create a unique and repeatable 9-digit "Meet Code".
- **Function** `get_lan_ip()`**:** A robust function to find the host's local LAN IP address, with a fallback for offline environments.

5.2.4 Shared Utilities and Scripts

- `generate_certs.py`**:** Standalone script (run on server) to generate the `server.crt` and `server.key` files.